

Mata Kuliah Workshop Pemrograman Framework
LAPORAN E - GOVERNANCE
Sistem Pengajuan Subsidi Pupuk Kabupaten Mojokerto



Dosen Pengampu:
Amma Liesvarastranta Haz, S.Tr.T., M.T.

Disusun Oleh:
Syarifatullah Ceva Efendy (312521002)
Selby Wafi Nurjuan (3125521018)

PROGRAM STUDI D3 TEKNIK INFORMATIKA PSDKU LAMONGAN
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

E-Pupuk: Sistem Pengajuan Subsidi Pupuk Kabupaten Mojokerto Berbasis Data Terbuka Pemerintah

Syarifatullah Ceva Efendy*, Selby Wafi Nurjuan[†]

Program Studi D3 Teknik Informatika PSDKU Lamongan

Politeknik Elektronika Negeri Surabaya

Email: *312521002@student.pens.ac.id, [†]3125521018@student.pens.ac.id

Abstract—Pengelolaan dan penyaluran pupuk bersubsidi merupakan aspek krusial dalam mendukung produktivitas sektor pertanian di Kabupaten Mojokerto. Namun, proses rekapitulasi data dan pengajuan manual yang belum terintegrasi sering menimbulkan kendala seperti keterlambatan informasi, kesalahan pencatatan, serta ketidaksesuaian alokasi kuota. Penelitian ini mengusulkan E-Pupuk, sebuah sistem informasi berbasis web yang mengintegrasikan pendataan petani, pemeriksaan kuota RDKK, verifikasi oleh Petugas Penyuluh Lapangan (PPL), pencatatan realisasi distribusi, hingga pemantauan oleh Dinas Pertanian. Dengan memanfaatkan data terbuka pemerintah (open data) sebagai acuan awal, E-Pupuk meningkatkan efisiensi administrasi, transparansi informasi, serta akurasi penyaluran pupuk bersubsidi agar lebih tepat sasaran.

Index Terms—e-Governance, E-Pupuk, Subsidi Pupuk, Open Data, RDKK, Kabupaten Mojokerto.

I. PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi dan komunikasi mendorong transformasi tata kelola pemerintahan menuju *e-Governance*. Perubahan ini menuntut pelayanan publik yang lebih transparan, efisien, dan akuntabel melalui pemanfaatan teknologi serta pengelolaan data secara terintegrasi. Salah satu penerapannya adalah pemanfaatan data terbuka pemerintah (*open data*) sebagai sumber data yang dapat mendukung proses pelayanan publik secara aktif [1].

Dalam sektor pertanian, pemerintah Kabupaten Mojokerto menyalurkan program pupuk bersubsidi untuk membantu petani menjaga produktivitas dan mengurangi biaya produksi. Alokasi pupuk subsidi mengacu pada Rencana Definitif Kebutuhan Kelompok (RDKK) yang disusun oleh kelompok tani dan diteruskan kepada Dinas Pertanian. Namun, proses pengelolaan dan rekapitulasi data yang masih belum terintegrasi dapat menyebabkan keterlambatan penyampaian informasi, kesalahan pencatatan, serta kesulitan dalam melakukan verifikasi pengajuan pupuk [2].

Kondisi tersebut juga menyebabkan petani kesulitan memperoleh informasi mengenai sisa kuota kelompok dan status pengajuan mereka. Di sisi lain, Petugas Penyuluh Lapangan (PPL) membutuhkan waktu dalam melakukan verifikasi pengajuan, sedangkan pihak Dinas Pertanian memerlukan sistem

yang dapat membantu memantau realisasi penyaluran pupuk secara lebih cepat dan terstruktur. Oleh karena itu, diperlukan sistem yang mampu mengintegrasikan proses pengajuan, verifikasi, distribusi, serta pemantauan data pupuk bersubsidi [3].

Sebagai solusi, dikembangkan E-Pupuk – Sistem Pengajuan Subsidi Pupuk Kabupaten Mojokerto, yaitu sistem berbasis web yang mengintegrasikan pendataan petani dan lahan, pengajuan pupuk, pemeriksaan kuota RDKK, verifikasi oleh PPL, pencatatan realisasi distribusi, serta penyusunan laporan. Sistem ini diharapkan dapat meningkatkan efisiensi proses administrasi, mengurangi kesalahan data, meningkatkan transparansi informasi, dan membantu mewujudkan distribusi pupuk bersubsidi yang lebih tepat sasaran di Kabupaten Mojokerto [4].

B. Rumusan Masalah

- 1) Bagaimana mengelola data petani dan kelompok tani penerima pupuk subsidi secara terpusat?
- 2) Bagaimana menentukan kewajaran kuota pengajuan pupuk berdasarkan data RDKK?
- 3) Bagaimana membantu petugas penyuluh melakukan verifikasi pengajuan secara cepat dan objektif?
- 4) Bagaimana petani dapat mengetahui status pengajuan dan distribusi pupuknya?
- 5) Bagaimana Dinas Pertanian dapat memantau realisasi distribusi pupuk subsidi secara cepat dan terstruktur?

C. Tujuan

- 1) Membuat sistem pengelolaan data petani dan kelompok tani secara terintegrasi.
- 2) Membantu proses verifikasi pengajuan berdasarkan data RDKK dan luas lahan.
- 3) Mempermudah petani memperoleh informasi status pengajuan dan distribusi pupuk.
- 4) Menyediakan dashboard monitoring dan pelaporan bagi Dinas Pertanian.

ACKNOWLEDGMENT

Penulis menyampaikan terima kasih kepada Dosen Pengampu Mata Kuliah Workshop Pemrograman Framework, Bapak Amma Liesvarastranta Haz, S.Tr.T., M.T., atas bimbingan dan arahan selama penyusunan laporan ini, serta Politeknik Elektronika Negeri Surabaya.

REFERENCES

- [1] R. Heeks, "Implementing and Managing e-Government: An International Text," SAGE Publications, 2006.
- [2] Kementerian Pertanian Republik Indonesia, "Pedoman Pengelolaan Sistem Elektronik Rencana Definitif Kebutuhan Kelompok (e-RDKK)," Jakarta, 2021.
- [3] Dinas Pertanian Kabupaten Mojokerto, "Laporan Evaluasi Penyaluran Pupuk Bersubsidi Kabupaten Mojokerto," 2024.
- [4] A. S. Ahmad, "Pemanfaatan Open Government Data untuk Transparansi Layanan Publik di Sektor Pertanian," *Jurnal Teknologi dan Governance*, vol. 8, no. 1, pp. 45–56, 2023.

II. SUMBER KODE

Bab ini menyajikan kode sumber utama sistem E-Pupuk yang terdiri dari komponen *frontend* berbasis React dan komponen *backend* berbasis Python FastAPI. Kode yang ditampilkan merupakan potongan representatif dari setiap modul utama sistem.

A. Frontend — App.jsx (Routing Utama)

File App.jsx merupakan komponen root aplikasi React yang mendefinisikan seluruh struktur routing, komponen proteksi akses berdasarkan role (*Protected Route*), serta tata letak halaman dengan Navbar.

Listing 1. frontend/src/App.jsx

```
1 import React, { useEffect } from 'react';
2 import { BrowserRouter, Routes, Route,
3       Navigate } from 'react-router-dom';
4 import { Toaster } from 'react-hot-toast';
5 import { useAuthStore } from './store/authStore';
6 import Navbar from './components/Navbar';
7 import Landing from './pages/Landing';
8 import Register from './pages/Register';
9 import Login from './pages/Login';
10 import PetaniDashboard from './pages/PetaniDashboard';
11 import AdminDashboard from './pages/AdminDashboard';
12 import PPLDashboard from './pages/PPLDashboard';
13 import QRScannerKiosk from './pages/QRScannerKiosk';
14
15 function ProtectedRoute({ children, allowedRoles }) {
16   const { isAuthenticated, user, isLoading } = useAuthStore();
17   if (isLoading) return <div className="loader" />;
18   if (!isAuthenticated || !user)
19     return <Navigate to="/login" replace />;
20   if (allowedRoles && !allowedRoles.includes(user.role)) {
21     if (user.role === 'ADMIN') return <Navigate to="/admin" replace />;
22     if (user.role === 'PPL') return <Navigate to="/ppl" replace />;
23     return <Navigate to="/petani" replace />;
24   }
25   return children;
26 }
27
28 function DashboardLayout({ children }) {
29   return (
30     <div className="min-h-screen bg-slate-50 flex flex-col">
31       <Navbar />
32       <main className="flex-1 max-w-7xl mx-auto w-full
33         px-4 sm:px-6 lg:px-8 pt-6">
34         {children}
35       </main>
36     </div>
37   );
38 }
39
40 export default function App() {
41   const { initAuth } = useAuthStore();
42   useEffect(() => { initAuth(); }, [initAuth]);
43
44   return (
45     <BrowserRouter>
46       <Toaster position="top-right" />
47       <Routes>
48         <Route path="/" element={<Landing />} />
49         <Route path="/login" element={<Login />} />
50         <Route path="/login/:portalRole" element={<Login />} />
51         <Route path="/register" element={<Register />} />
52         <Route path="/kiosk-scanner" element={<QRScannerKiosk />} />
53
54         <Route path="/petani" element={
55           <ProtectedRoute allowedRoles={['PETANI', 'ADMIN']}>
56             <DashboardLayout><PetaniDashboard /></DashboardLayout>
57           </ProtectedRoute>
58         } />
59         <Route path="/admin" element={
60           <ProtectedRoute allowedRoles={['ADMIN']}>
61             <DashboardLayout><AdminDashboard /></DashboardLayout>
62           </ProtectedRoute>
63         } />
64         <Route path="/ppl" element={
65           <ProtectedRoute allowedRoles={['PPL', 'ADMIN']}>
66             <DashboardLayout><PPLDashboard /></DashboardLayout>
67           </ProtectedRoute>
68         } />
69         <Route path="*" element={<Navigate to="/" replace />} />
70       </Routes>

```

```
71     </BrowserRouter>
72   );
73 }
```

B. Backend — main.py (Entry Point FastAPI)

File main.py adalah titik masuk aplikasi backend FastAPI. Di sini didaftarkan semua router, middleware CORS, dan endpoint root.

Listing 2. backend/app/main.py

```
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from fastapi.staticfiles import StaticFiles
4 from app.core.config import settings
5 from app.routers import (
6     auth, farmers, lands, applications,
7     admin, ppl, distributions, files
8 )
9
10 app = FastAPI(
11     title=settings.PROJECT_NAME,
12     description="Sistem Verifikasi & Distribusi Pupuk "
13     "Bersubsidi Kabupaten Mojokerto",
14     version="1.0.0"
15 )
16
17 app.add_middleware(
18     CORSMiddleware,
19     allow_origins=["*"],
20     allow_credentials=True,
21     allow_methods=["*"],
22     allow_headers=["*"],
23 )
24
25 app.include_router(auth.router, prefix=settings.API_V1_STR)
26 app.include_router(farmers.router)
27 app.include_router(lands.router)
28 app.include_router(applications.router)
29 app.include_router(admin.router)
30 app.include_router(ppl.router)
31 app.include_router(distributions.router)
32 app.include_router(files.router)
33
34 @app.get("/")
35 def root():
36     return {"app": settings.PROJECT_NAME,
37           "status": "online",
38           "docs_url": "/docs"}
39
40 @app.get("/api/health")
41 def health_check():
42     return {"status": "ok", "service": "epupuk-backend"}
```

C. Backend — routers/auth.py (Autentikasi JWT)

Modul ini menangani proses login, pengambilan profil pengguna aktif, dan logout. Token JWT dibuat menggunakan python-jose.

Listing 3. backend/app/routers/auth.py

```
1 from fastapi import APIRouter, Depends, HTTPException, status
2 from sqlalchemy.orm import Session
3 from app.core.database import get_db
4 from app.core.security import verify_password, create_access_token
5 from app.core.dependencies import get_current_user, log_audit_action
6 from app.models.models import User, Farmer
7 from app.schemas.schemas import LoginRequest, Token, UserResponse
8
9 router = APIRouter(prefix="/auth", tags=["Authentication"])
10
11 @router.post("/login", response_model=Token)
12 def login(request: LoginRequest, db: Session = Depends(get_db)):
13     user = db.query(User).filter(
14         User.username == request.username).first()
15     if not user or not verify_password(
16         request.password, user.password_hash):
17         raise HTTPException(
18             status_code=status.HTTP_401_UNAUTHORIZED,
19             detail="Username atau kata sandi tidak valid.")
20     if not user.is_active:
21         raise HTTPException(
22             status_code=status.HTTP_403_FORBIDDEN,
23             detail="Akun ini telah dinonaktifkan.")
24
25     farmer_id, display_name = None, user.username
26     if user.role == "PETANI":
27         farmer = db.query(Farmer).filter(
28             Farmer.user_id == user.id).first()
29         if farmer:
30             farmer_id = farmer.id
31             display_name = farmer.nama
32
33     token = create_access_token(data={
34         "user_id": user.id,
35         "username": user.username,
36         "role": user.role
37     })
38     log_audit_action(db, user_id=user.id,
39                     action="LOGIN", resource="auth",
40                     details=f"User {user.username} logged in")
41     return Token(access_token=token, token_type="bearer",
42                 role=user.role, user_id=user.id,
43                 username=user.username, nama=display_name,
44                 farmer_id=farmer_id)
45
46 @router.get("/me")
47 def get_current_profile(
48     current_user: User = Depends(get_current_user),
49     db: Session = Depends(get_db)
50 ):
51     farmer_id, farmer_data = None, None
52     if current_user.role == "PETANI":
53         farmer = db.query(Farmer).filter(
54             Farmer.user_id == current_user.id).first()
55         if farmer:
56             farmer_id = farmer.id
57             farmer_data = {"id": farmer.id,
58                           "nama": farmer.nama,
59                           "nik": farmer.nik}
60     return {"id": current_user.id,
61           "username": current_user.username,
62           "role": current_user.role,
63           "farmer_id": farmer_id,
64           "farmer": farmer_data}
65
66 @router.post("/logout")
67 def logout(current_user: User = Depends(get_current_user),
68            db: Session = Depends(get_db)):
69     log_audit_action(db, user_id=current_user.id,
70                     action="LOGOUT", resource="auth")
71     return {"message": "Berhasil keluar dari sistem"}
```

D. Backend — routers/applications.py (Pengajuan Subsidi)

Modul ini menangani seluruh siklus pengajuan pupuk bersubsidi, mulai dari pembuatan pengajuan baru oleh petani, pembacaan daftar pengajuan berdasarkan role, hingga statistik dashboard.

Listing 4. backend/app/routers/applications.py (notongan)

```
1 from typing import List, Optional
2 from fastapi import APIRouter, Depends, HTTPException, status
3 from sqlalchemy.orm import Session, joinedload
4 from app.core.database import get_db
5 from app.core.dependencies import get_current_user, log_audit_action
6 from app.models.models import (
7     User, Farmer, Land, Application, ApplicationStatus,
8     Fertilizer, Notification, FieldSurvey)
9 from app.schemas.schemas import (
10     ApplicationCreate, ApplicationResponse, DashboardStatsPetani)
11
12 router = APIRouter(prefix="/api/applications",
13                     tags=["Applications"])
14
15 @router.get("", response_model=List[ApplicationResponse])
16 def get_applications(
17     status_filter: Optional[str] = None,
18     farmer_id: Optional[int] = None,
19     current_user: User = Depends(get_current_user),
20     db: Session = Depends(get_db)
21 ):
22     query = db.query(Application).options(
23         joinedload(Application.farmer),
24         joinedload(Application.land),
25         joinedload(Application.fertilizer),
26         joinedload(Application.survey)
27     )
28     if current_user.role == "PETANI":
29         farmer = db.query(Farmer).filter(
30             Farmer.user_id == current_user.id).first()
31         if not farmer:
32             return []
33         query = query.filter(
34             Application.farmer_id == farmer.id)
35     elif current_user.role == "PPL":
36         query = query.filter(
37             Application.assigned_ppl_id == current_user.id)
38     elif farmer_id:
39         query = query.filter(Application.farmer_id == farmer_id)
40
41     if status_filter:
42         query = query.filter(Application.status == status_filter)
43
44     return query.order_by(Application.created_at.desc()).all()
```

E. Backend — routers/ppl.py (Survei Lapangan)

Modul PPL menangani penugasan survei, memulai survei, serta pengiriman laporan hasil survei lapangan oleh Petugas Penyuluh Lapangan. Status pengajuan berubah otomatis mengikuti alur survei.

Listing 5. backend/app/routers/ppl.py (potongan)

```
1 from typing import List
2 from fastapi import APIRouter, Depends, HTTPException, status
3 from sqlalchemy.orm import Session, joinedload
4 from app.core.database import get_db
5 from app.core.dependencies import (
6     get_current_user, require_role, log_audit_action)
7 from app.models.models import (
8     User, Application, ApplicationStatus,
9     FieldSurvey, Notification)
10 from app.schemas.schemas import (
11     ApplicationResponse, FieldSurveyCreate, FieldSurveyResponse)
12
13 router = APIRouter(prefix="/api/ppl",
14                     tags=["PPL Field Survey"])
15
16 @router.get("/assigned-tasks",
17             response_model=List[ApplicationResponse])
18 def get_assigned_tasks(
19     current_user: User = Depends(require_role("PPL", "ADMIN")),
20     db: Session = Depends(get_db)
21 ):
22     query = db.query(Application).options(
23         joinedload(Application.farmer),
24         joinedload(Application.land),
25         joinedload(Application.fertilizer),
26         joinedload(Application.survey))
27     if current_user.role == "PPL":
28         query = query.filter(
29             Application.assigned_ppl_id == current_user.id)
30     query = query.filter(Application.status.in_(
31         ApplicationStatus.DITUGASKAN_KE_PPL,
32         ApplicationStatus.SURVEI_LAPANGAN,
33         ApplicationStatus.MENUNGGU_PERSETUJUAN_AKHIR))
34     return query.order_by(Application.created_at.desc()).all()
35
36 @router.post("/applications/{id}/start-survey")
37 def start_survey(
38     id: int,
39     current_user: User = Depends(require_role("PPL", "ADMIN")),
40     db: Session = Depends(get_db)
41 ):
42     app = db.query(Application).filter(
43         Application.id == id).first()
44     if not app:
45         raise HTTPException(status_code=404,
46                             detail="Pengajuan tidak ditemukan")
47     if (current_user.role == "PPL" and
48         app.assigned_ppl_id != current_user.id):
49         raise HTTPException(status_code=403,
50                             detail="Tugas ini bukan untuk Anda")
51     app.status = ApplicationStatus.SURVEI_LAPANGAN
52     db.commit(); db.refresh(app)
53     return {"message": "Status survei lapangan dimulai",
54           "status": app.status}
```

F. Backend — routers/admin.py (Manajemen Admin)

Modul Admin menyediakan endpoint untuk statistik dashboard, verifikasi berkas pengajuan, penugasan PPL, serta persetujuan akhir distribusi pupuk bersubsidi.

Listing 6. backend/app/routers/admin.py (potongan)

```
1 from decimal import Decimal
2 from fastapi import APIRouter, Depends
3 from sqlalchemy.orm import Session
4 from sqlalchemy import func
5 from app.core.database import get_db
6 from app.core.dependencies import require_role, log_audit_action
7 from app.models.models import (
8     User, Farmer, Land, Application, ApplicationStatus,
9     Distribution, Notification, AuditLog, FieldSurvey)
10 from app.schemas.schemas import (
11     DashboardStatsAdmin, ApplicationAdminVerifyRequest,
12     ApplicationAssignPPLRequest, ApplicationFinalApproveRequest)
13
14 router = APIRouter(prefix="/api/admin",
15                    tags=["Admin Management"])
16
17 @router.get("/stats", response_model=DashboardStatsAdmin)
18 def get_admin_stats(
19     current_user: User = Depends(require_role("ADMIN")),
20     db: Session = Depends(get_db)
21 ):
22     total = db.query(Application).count()
23     menunggu = db.query(Application).filter(
24         Application.status.in_(
25             ApplicationStatus.DIAJUKAN,
26             ApplicationStatus.MENUNGGU_VERIFIKASI_BERKAS
27         )).count()
28     perlu_ppl = db.query(Application).filter(
29         Application.status.in_(
30             ApplicationStatus.BERKAS_TERVERIFIKASI,
31             ApplicationStatus.DITUGASKAN_KE_PPL,
32             ApplicationStatus.SURVEI_LAPANGAN
33         )).count()
34     mng_akhir = db.query(Application).filter(
35         Application.status ==
36         ApplicationStatus.MENUNGGU_PERSETUJUAN_AKHIR
37     ).count()
38     disetujui = db.query(Application).filter(
39         Application.status.in_(
40             ApplicationStatus.DISETUIJUI,
41             ApplicationStatus.DIJADWALKAN_DISTRIBUSI,
42             ApplicationStatus.TERSALURKAN
43         )).count()
44     ditolak = db.query(Application).filter(
45         Application.status.in_(
46             ApplicationStatus.DITOLAK_BERKAS,
47             ApplicationStatus.DITOLAK_LAPANGAN
48         )).count()
49     total_petani = db.query(Farmer).count()
50     total_lahan = db.query(
51         func.sum(Land.luas_m2)).scalar() or Decimal("0")
52
53     return DashboardStatsAdmin(
54         total_pengajuan=total,
55         menunggu_verifikasi_berkas=menunggu,
56         perlu_survei_ppl=perlu_ppl,
57         menunggu_persetujuan_akhir=mng_akhir,
58         disetujui=disetujui, ditolak=ditolak,
59         total_petani=total_petani,
60         total_lahan_m2=total_lahan,
61         kuota_urea_kg=Decimal("150000.00"),
62         kuota_npk_kg=Decimal("120000.00"))
```

G. Database — *schema.sql* (Skema MySQL)

Skema database E-Pupuk terdiri dari tabel-tabel utama yang saling berelasi, mencakup entitas pengguna, petani, lahan, pengajuan, distribusi, hingga log audit.

Listing 7. backend/schema.sql

```
1 -- Database Schema: E-Pupuk Kabupaten Mojokerto
2 CREATE DATABASE IF NOT EXISTS epupuk
3 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
4 USE epupuk;
5
6 -- 1. Pengguna
7 CREATE TABLE users (
8   id INT AUTO_INCREMENT PRIMARY KEY,
9   username VARCHAR(100) UNIQUE NOT NULL,
10  email VARCHAR(150) UNIQUE NOT NULL,
11  password_hash VARCHAR(255) NOT NULL,
12  role ENUM('PETANI', 'PPL', 'ADMIN') NOT NULL,
13  is_active BOOLEAN DEFAULT TRUE,
14  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
16 COLLATE=utf8mb4_unicode_ci;
17
18 -- 2. Kelompok Tani
19 CREATE TABLE farmer_groups (
20   id INT AUTO_INCREMENT PRIMARY KEY,
21   nama_kelompok VARCHAR(150) NOT NULL,
22   ketua_id INT NULL,
23   wilayah VARCHAR(200) NOT NULL,
24   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
25 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
26
27 -- 3. Profil Petani
28 CREATE TABLE farmers (
29   id INT AUTO_INCREMENT PRIMARY KEY,
30   user_id INT UNIQUE NOT NULL,
31   nama VARCHAR(150) NOT NULL,
32   nik VARCHAR(20) UNIQUE NOT NULL,
33   kontak VARCHAR(20),
34   alamat TEXT NOT NULL,
35   foto_ktp_url VARCHAR(500),
36   tanggal_registrasi DATE DEFAULT (CURRENT_DATE),
37   farmer_group_id INT NULL,
38   FOREIGN KEY (user_id) REFERENCES users(id)
39   ON DELETE CASCADE,
40   FOREIGN KEY (farmer_group_id)
41     REFERENCES farmer_groups(id) ON DELETE SET NULL
42 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
43
44 -- 4. Lahan Pertanian
45 CREATE TABLE lands (
46   id INT AUTO_INCREMENT PRIMARY KEY,
47   farmer_id INT NOT NULL,
48   lokasi_deskripsi VARCHAR(300),
49   alamat_lahan TEXT NOT NULL,
50   latitude DECIMAL(10,7),
51   longitude DECIMAL(10,7),
52   luas_m2 DECIMAL(12,2) NOT NULL,
53   commodity_id INT,
54   status_kepemilikan ENUM('MILIK', 'GARAP', 'SEWA')
55     DEFAULT 'MILIK',
56   foto_lahan_url VARCHAR(500),
57   FOREIGN KEY (farmer_id) REFERENCES farmers(id)
58   ON DELETE CASCADE
59 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
60
61 -- 5. Pengajuan Subsidi
62 CREATE TABLE applications (
63   id INT AUTO_INCREMENT PRIMARY KEY,
64   farmer_id INT NOT NULL,
65   land_id INT NOT NULL,
66   fertilizer_id INT NOT NULL,
67   jumlah_diajukan DECIMAL(10,2) NOT NULL,
68   kuota_maksimal DECIMAL(10,2),
69   status ENUM(
70     'DIAJUKAN', 'MENUNGGU_VERIFIKASI_BERKAS',
71     'PERLU_PERBAIKAN_BERKAS', 'BERKAS_TERVERIFIKASI',
72     'DITUGASKAN_KE_PPL', 'SURVEI_LAPANGAN',
73     'MENUNGGU_PERSETUJUAN_AKHIR', 'DISETUIJUI',
74     'DITOLAK_BERKAS', 'DITOLAK_LAPANGAN',
75     'DIJADWALKAN_DISTRIBUSI', 'TERSALURKAN'
76 ) DEFAULT 'MENUNGGU_VERIFIKASI_BERKAS',
```

```
77 assigned_ppl_id INT NULL,
78 tanggal_pengajuan TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
79 FOREIGN KEY (farmer_id) REFERENCES farmers(id),
80 FOREIGN KEY (land_id) REFERENCES lands(id),
81 FOREIGN KEY (assigned_ppl_id)
82 REFERENCES users(id) ON DELETE SET NULL
83 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
84
85 -- 6. Distribusi & QR Code
86 CREATE TABLE distributions (
87 id INT AUTO_INCREMENT PRIMARY KEY,
88 application_id INT NOT NULL,
89 qr_hash VARCHAR(255) UNIQUE NOT NULL,
90 kuota_disetujui DECIMAL(10,2),
91 tanggal_jadwal DATE,
92 status ENUM('TERJADWAL','TERSALURKAN')
93 DEFAULT 'TERJADWAL',
94 FOREIGN KEY (application_id)
95 REFERENCES applications(id)
96 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

III. DESIGN SYSTEM

Design system E-Pupuk merancang visual konsisten, komponen antarmuka modular, skema warna yang harmonis, dan elemen navigasi intuitif untuk pengguna dari berbagai peran (Petani, PPL, Admin, dan Pimpinan Dinas).

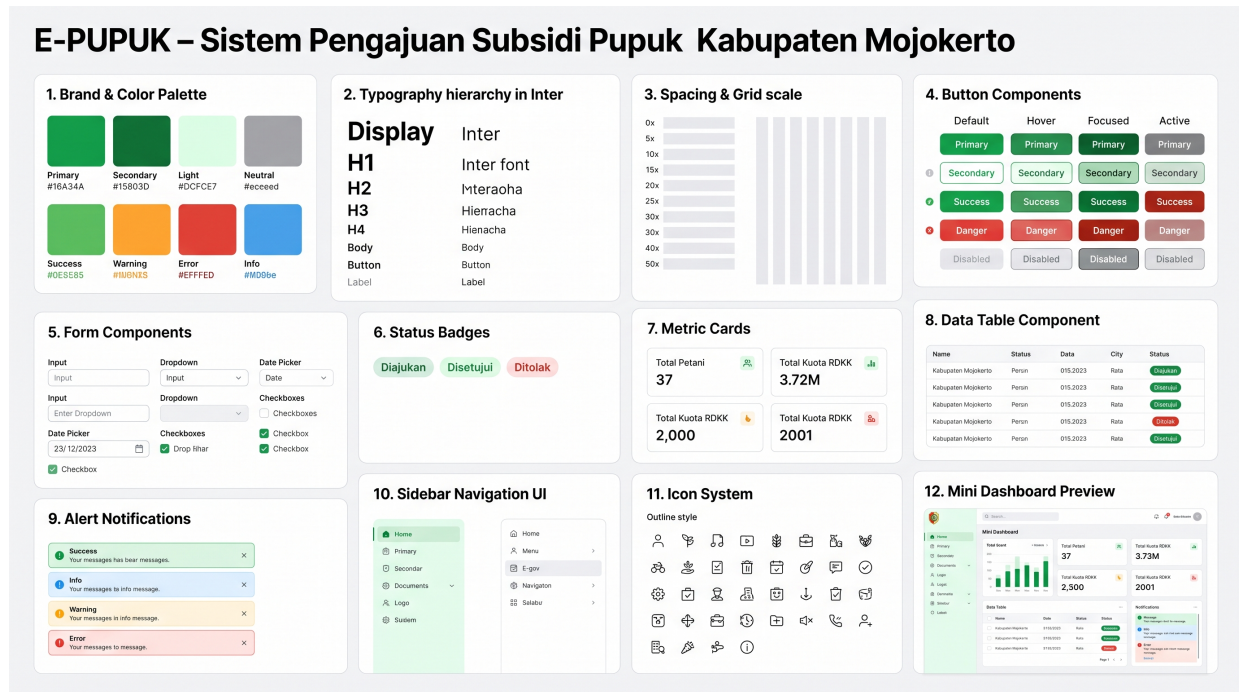


Fig. 1. Design System E-Pupuk

IV. ARSITEKTUR SISTEM

Arsitektur sistem E-Pupuk mengintegrasikan komponen frontend (React Web App), backend REST API (Python FastAPI), serta basis data terpusat (MySQL). Data terbuka pemerintah dimanfaatkan melalui modul data-seeding untuk memberikan informasi baseline kuota RDKK dan luas lahan.

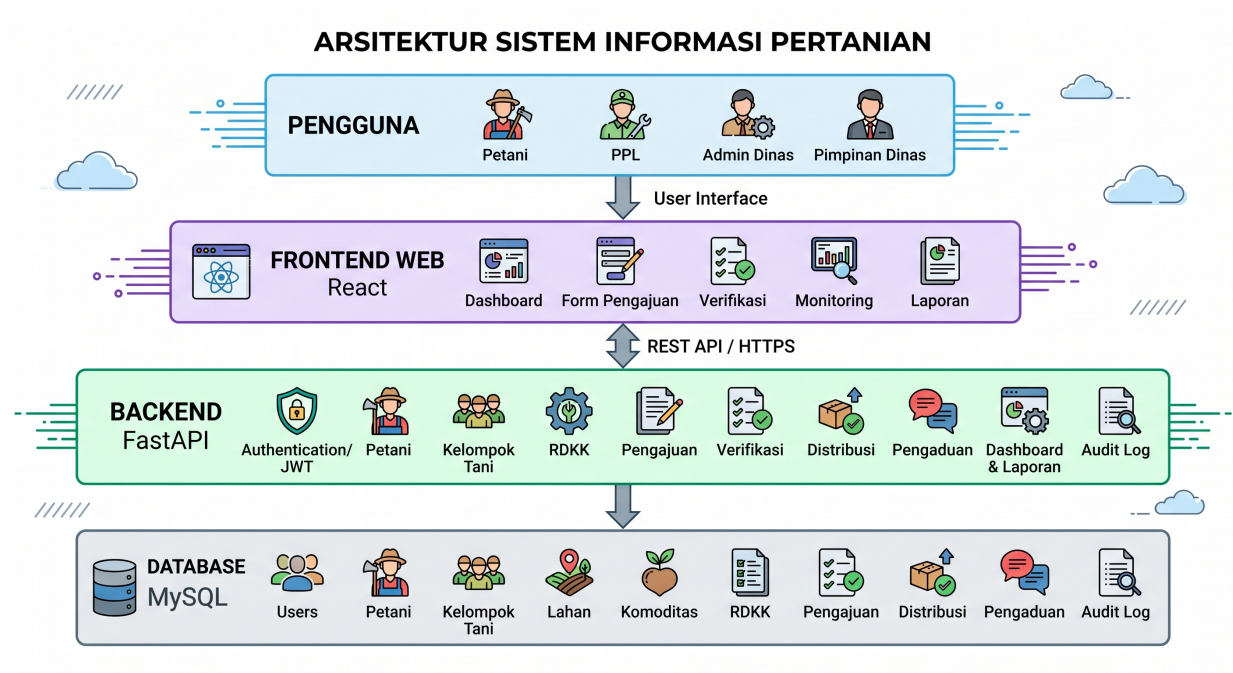


Fig. 2. Arsitektur Sistem E-Pupuk

V. MOCKUP UI

Bab ini menyajikan 5 rancangan antarmuka (*Mockup UI*) utama sistem E-Pupuk dari folder *Frontend* yang mencakup alur proses pengguna dari halaman utama, pengajuan petani, verifikasi PPL, hingga monitoring eksekutif Dinas Pertanian.

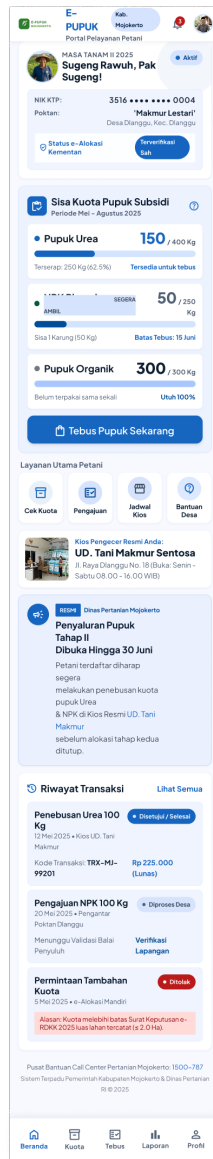


Fig. 3. Mockup UI – Halaman Utama / Beranda E-Pupuk Kabupaten Mojokerto



Fig. 4. Mockup UI – Dashboard Petani (Informasi Kuota RDKK dan Status Lahan)

Syarifatullah C.

Petani Poktan

←

Ajukan Kebutuhan Pupuk

MT-12025 • Poktan Tani Makmur

Petani Pemohon Terdaftar

Syarifatullah Ceva Efendy

Valid

Luas Lahan

0,5 Ha (5.000 m²)

Komoditas & Wilayah

Padi Sawah • Puri

Pilih Jenis Pupuk Bersubsidi

Wajib

Urea Bersubsidi

▼

HET Resmi Pemerintah:

Rp 2.250 / kg

Batas Kuota Lahan Anda: 125 kg

Tersedia untuk alokasi musim tanam saat ini sesuai RDKK Desa Puri

Jumlah Pengajuan (kg)

Kelipatan 5 kg

−

100

kg

+

Gunakan tombol atau ketik langsung

Status: Sesuai Kuota Lahan

Pengajuan dalam batas wajar sistem e-Alokasi Mojokerto

Estimasi Total Biaya Tebus

100 kg × Rp 2.250

Rp 225.000

Catatan Tambahan untuk PPL

Opsional

Pilih cepat kebutuhan pemupukan:

+ Pemupukan awal (Dasar)

+ Pemupukan susulan I

+ Pemupukan susulan II

+ Percepatan masa tanam

Tuliskan catatan khusus atau rencana tanggal tebus di Kios Puri...

Batal

Kirim Pengajuan Pupuk

+ Ajukan Pupuk

Beranda

Ajukan

Riwayat

Pengaduan

Profil

Fig. 5. Mockup UI – Form Pengajuan Kebutuhan Pupuk Subsidi oleh Petani

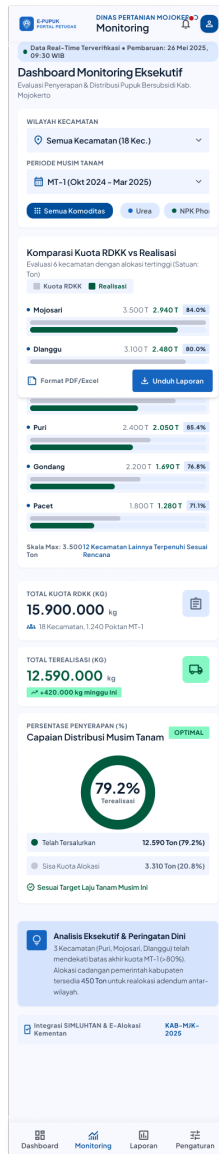


Fig. 7. Mockup UI – Dashboard Monitoring Eksekutif Dinas Pertanian